TECHNICAL PROJECT REPORT

CryptoGuard

Cryptographic Inventory and Post-Quantum Migration Planning

REPORT TYPE
Technical Project Report

PROJECT STATUS
Prototype; benchmark preliminary

PROJECT FOCUS
Cryptographic Inventory and Migration Readiness

IMPLEMENTATION
Python, CSV-based inventory analysis, risk triage, migration recommendations, and Streamlit dashboard

REPORT DATE
11 October 2026

PS-P2: Post-Quantum Cryptography P2 — Crypto Inventory and Migration Planner

Executive Summary

CryptoGuard is a prototype designed to help organisations understand where cryptographic algorithms are used, identify assets that may need attention as quantum computing advances, and organise a practical migration plan. The current implementation combines a structured cryptographic inventory, an initial risk-scoring method, algorithm-specific migration guidance, and a dashboard for reviewing results.

The sample inventory contains five cryptographic assets. RSA, ECDSA, and ECDH are labelled High risk by the prototype, while 3DES is labelled Review and ML-KEM is classified as PQC. These labels are intended to support initial prioritisation. They are not official NIST risk ratings and do not replace a complete cryptographic security assessment.

A separate preliminary circuit-compilation experiment compared two illustrative processor layouts. The same five-logical-qubit test circuit compiled to a depth of 24 on the five-qubit baseline and a depth of 21 on the seven-qubit heavy-hex-inspired layout. Both compiled circuits had seven two-qubit gates and zero SWAPs in the recorded run. These values describe software transpilation, not execution on quantum hardware.

The official processor definitions were not available in the challenge materials reviewed. Therefore, the benchmark remains preliminary. A valid challenge comparison requires the organiser-defined processor configurations and a workload and metrics that match the challenge requirements.

1. Project Objectives

CryptoGuard aims to provide a practical starting point for cryptographic inventory and post-quantum migration planning.

The main objectives are:

- Create a consistent inventory of cryptographic assets and the algorithms protecting them.
- Identify algorithms and uses that warrant closer review for post-quantum migration.
- Attach practical migration guidance to inventory findings.
- Present results in a clear format that supports technical review and planning.
- Establish a reproducible benchmarking workflow that can later use the official processor definitions.

2. System Overview

The current workflow begins with a CSV inventory. The program validates the required fields, assesses each algorithm, calculates a prototype score, and adds migration categories and recommendations. The processed records are saved to a CSV file and can be reviewed through the Streamlit dashboard.

2.1 System Workflow

Stage	Purpose
Inventory Input	Reads asset identifiers, names, algorithms, data sensitivity, and exposure from a CSV file.
Validation	Checks that the expected inventory columns are present before processing.
Risk Assessment	Classifies supported algorithms and calculates an illustrative triage score.
Migration Planning	Adds a migration category, suggested action, and explanatory recommendation.
Output and Review	Writes the processed inventory to `data/processed/risk_assessment.csv` and supports dashboard review.

2.2 Technology and Project Structure

The project is implemented in Python. It uses a CSV-based inventory pipeline, separate modules for risk scoring and migration recommendations, and Streamlit for visualising the results.

The benchmark module uses Qiskit to create and transpile a test circuit against configured processor layouts. The current benchmark tests the compilation workflow rather than the execution of a cryptographic workload on actual quantum hardware.

3. Cryptographic Inventory Results

The sample inventory demonstrates how CryptoGuard processes different algorithm families and generates a consistent summary. Five records are sufficient to demonstrate the current workflow, but they do not represent a complete enterprise cryptographic inventory.

3.1 Current Risk Assessment

Asset ID	Algorithm	Risk Label	Score	Triage Interpretation
A001	RSA	High	100	Prioritise discovery of public-key uses and plan migration.
A002	ECDSA	High	87	Review signature, certificate, identity, and dependent workflows.
A003	3DES	Review	70	Review legacy use and replacement needs against applicable policy.
A004	ECDH	High	100	Prioritise key-establishment dependencies for post-quantum transition.
A005	ML-KEM	PQC	27	Recognised by the prototype as a post-quantum key-encapsulation algorithm.

Interpretation note: These scores are outputs of the current prototype and are not official NIST scores. A low score does not guarantee that an implementation is secure. Algorithm parameters, software quality, key management, protocol design, and deployment practices still require review.

3.2 Interpretation of the Inventory

RSA and ECDH are examples of conventional public-key cryptographic techniques that require particular attention when preparing for cryptographically relevant quantum computing. ECDSA is a conventional digital-signature algorithm that should also be included in post-quantum migration planning.

The 3DES entry is classified as Review in the current prototype. This should not be interpreted as a quantum-specific risk rating. Its suitability must be assessed against applicable cryptographic standards, organisational policy, and the context in which it is used.

ML-KEM is recognised as a post-quantum key-encapsulation algorithm. Its presence does not automatically establish that an asset is secure. The implementation, parameter set, key management, and integration with the surrounding system still need to be checked.

4. Risk-Scoring Methodology

The current code maps supported algorithm names to a category and numeric score. Normalising common names, such as AES variants, helps the inventory treat related algorithms consistently.

This lightweight method provides a first-pass view of which records should be reviewed. However, an algorithm name alone cannot describe the complete security posture of an asset.

A stronger production model should use documented and consistently applied criteria. Relevant factors may include:

- Cryptographic purpose and algorithm parameters.
- Data sensitivity and the length of time data must remain confidential.
- External exposure and business criticality.

- Dependencies on applications, protocols, certificates, and libraries.
- Migration difficulty and compatibility requirements.
- Availability of supported replacement algorithms.

Any scoring weights should be explained and validated. The score should be treated as a prioritisation aid rather than a universal security formula.

5. Migration Planning Approach

CryptoGuard attaches a migration category, suggested action, and recommendation to each inventory record. This makes the output more useful than a risk label alone by helping teams identify what they should investigate next.

Recommendations should be reviewed by the asset owner and security team before any production change is made.

Area for Review	Planning Activity
RSA and ECDH uses	Locate all uses, map dependencies, identify long-lived sensitive data, and assess suitable post-quantum replacements.
ECDSA and other conventional signature uses	Inventory certificates, code signing, identity, and document-signing workflows; assess compatible post-quantum signature options.
Legacy symmetric algorithms such as 3DES	Review applicable standards and policy, identify compatibility constraints, and plan replacement where required.
PQC algorithms such as ML-KEM	Confirm standardised parameters, supported libraries, key management, interoperability, and secure deployment.

A practical migration programme should begin with discovery and dependency mapping. Teams should then prioritise assets based on risk and business impact, test replacement mechanisms in a controlled environment, validate interoperability, and plan deployment and rollback procedures.

The migration planner is intended to support this process. It should not automatically replace cryptographic algorithms without technical review and testing.

6. Preliminary Processor A/B Benchmark

A small software experiment was implemented to test the benchmark workflow. It creates one circuit and transpiles the same circuit against two illustrative connectivity layouts. A fixed transpiler seed was used to make the comparison more repeatable.

The configuration templates describe Processor A as a five-qubit baseline and Processor B as a seven-qubit heavy-hex-inspired layout.

The official coupling maps, basis gates, noise assumptions, and other processor settings were not available in the supplied challenge documents. The demonstration therefore uses assumed connectivity and a common basis-gate set to exercise the software workflow only.

6.1 Recorded Benchmark Results

Metric	Processor A	Processor B
Qubits Available	5	7
Logical Qubits in Test Circuit	5	5
Transpiled Circuit Depth	24	21
Two-Qubit Gate Count	7	7
SWAP Count	0	0
Transpilation Time (seconds)	0.267770	0.007168
Result Status	PRELIMINARY_ILLUSTRATIVE	PRELIMINARY_ILLUSTRATIVE

These are the values recorded by the current preliminary software experiment. They should be preserved exactly when reproducing the report.

6.2 Interpretation of Results

In the recorded run, the illustrative Processor B layout produced a circuit depth of 21, compared with 24 for Processor A. Both compiled circuits had seven two-qubit gates and no SWAP gates.

For this particular circuit and the assumed connectivity layouts, the transpiler found a slightly shallower implementation on Processor B. This result is specific to the test circuit and configuration used.

Processor B also had a shorter measured transpilation time in this run. However, compilation timing can be affected by machine load, software state, and transpiler heuristics. A single run is not enough to conclude that one configuration is consistently faster. Repeated runs and appropriate summary statistics would be needed.

6.3 Limitations of the Results

The experiment does not measure:

- Execution fidelity.
- Application-level accuracy.
- Noisy-circuit performance.
- Runtime on physical quantum hardware.
- The performance of CryptoGuard's cryptographic inventory or migration-planning workflow.

The circuit is a generic test circuit, not a workload derived from the cryptographic inventory or migration-planning task.

The processor settings are illustrative rather than confirmed official challenge configurations. As a result, these findings must not be presented as official challenge benchmark results or as evidence that Processor B is generally superior to Processor A.

7. Benchmark Limitations and Required Follow-Up

Before completing the official benchmark, the following steps are required:

1. Obtain the official Processor A and Processor B definitions, including coupling maps, basis gates, noise assumptions, and compilation or execution settings.
2. Replace illustrative values and validate the loaded configurations before benchmarking.
3. Define a problem-specific workload that directly reflects the challenge objective.
4. Use consistent workloads, compilation settings, seeds, and measurement procedures for both processors.
5. Repeat timing trials and report an appropriate summary, such as the median and variability.
6. If execution or noisy simulation is required, record the noise model and shot count and report relevant output-quality metrics.
7. Save configuration files, raw results, software versions, and run metadata so that the experiment can be reproduced.

The current implementation is a useful starting point for the benchmark workflow, but it does not yet satisfy the need for a validated, problem-specific comparison using official processor configurations.

8. Project Files and Reproducibility

The project is implemented in Python and uses a CSV-based inventory pipeline. The main outputs currently recorded are the processed inventory assessment and the preliminary processor comparison.

File or Component	Role
`main.py`	Runs the inventory analysis workflow.

File or Component	Role
src/inventory.py	Loads, validates, and processes inventory records.
src/risk_scoring.py	Classifies supported algorithms and calculates prototype scores.
src/migration.py	Provides migration categories and recommendations.
src/benchmark.py	Runs the preliminary circuit-transpilation comparison.
processors/processor_A.json and processors/processor_B.json	Configuration templates; official processor settings are still needed.
data/processed/risk_assessment.csv	Stores the processed cryptographic inventory and risk results.
results/tables/processor_comparison.csv	Stores the preliminary A/B compilation metrics.

For a final submission, the project should record the Python and Qiskit versions, the exact configuration files used, the command used for each run, and a run identifier or timestamp.

Any assumed, simulated, or illustrative value should be labelled clearly. Keeping raw results and configuration files alongside the report will make the work easier to verify and reproduce.

9. Future Development

The following improvements would make CryptoGuard more useful and strengthen the quality of the technical evaluation.

Complete the official benchmark: Apply the organiser-provided configurations and required workload and metrics. Repeat the experiment under controlled conditions and record the results.

Strengthen risk scoring: Document the scoring criteria, distinguish quantum risk from other cryptographic concerns, and validate the results through expert review.

Expand the inventory: Add fields for cryptographic purpose, parameter set, library, owner, location, data lifetime, business criticality, and system dependencies.

Improve migration tracking: Add owners, target dates, dependencies, testing requirements, rollback plans, and migration status.

Improve reporting: Add filters, summary counts, migration-priority views, and exportable evidence that supports technical reviews and audits.

10. Conclusion

CryptoGuard demonstrates an end-to-end starting point for cryptographic asset inventory, preliminary quantum-risk triage, and migration planning. The current pipeline processes sample assets, assigns prototype labels and scores, and attaches migration guidance. The separate compilation experiment also shows that basic circuit metrics can be collected for two configured layouts.

The project remains a prototype. Its risk scores are not official security ratings, and its Processor A/B comparison is preliminary because it uses illustrative configurations and a generic test circuit.

A defensible challenge benchmark requires the official processor definitions, a problem-specific workload, consistent measurement procedures, and reproducible results. With these improvements, CryptoGuard can develop into a more complete tool for organising cryptographic discovery and supporting post-quantum migration decisions.

Technical Note

This report summarises the project files and outputs available on 11 October 2026. The algorithm classifications are high-level engineering guidance, not a substitute for current standards or organisation-specific policy.

Before production use, consult current NIST post-quantum cryptography standards and transition guidance, validate library and protocol support, and obtain a formal security review.

Project Status: Prototype; benchmark preliminary